Universidad Complutense de Madrid
MSc in Cybersecurity

Master's Thesis (Trabajo de Fin de Máster)

MalWhere

An Automated, Evidence-Audited Pipeline for Threat Intelligence Extraction from Malware Samples

---

Author: Martín López Aranburu

Supervisors: Prof. Javier Domínguez Gómez
Prof. Román Ramírez Giménez

Academic Year 2025–2026

Contents

Abstract

Security operations teams rely on structured threat intelligence (IOCs, MITRE ATT&CK mappings, STIX/MISP-ready artifacts) to act on malware findings quickly. Broad reputation aggregators such as VirusTotal deliver that structure at enormous scale but do not reconcile their many signals against each other, so producing a defensible, evidence-linked technique mapping for one specific sample usually still falls to a human reverse engineer tracing code and writing a report by hand. This thesis presents **MalWhere**, a modular pipeline that combines static reverse engineering and dynamic sandbox detonation, reconciles their findings into confidence-scored MITRE ATT&CK technique mappings, and exports the result as STIX 2.1 bundles and live MISP events.

The pipeline is validated against three malware families with distinct architectures: a ransomware binary (Akira); a multi-stage loader that drops a rootkit driver, an AV-killer, and a remote-access C2 client (RoningLoader); and a feature-rich .NET infostealer (WhiteSnakeStealer). Each is ground-truthed through function-level, Ghidra-verified manual reverse engineering rather than superficial scanning. Two methodological contributions extend beyond a conventional static+dynamic pipeline: a cross-source *and* cross-level confidence reconciliation model that treats a technique observed by both static and dynamic evidence, or by a parent and a sub-technique from different sources, as stronger signal than either alone; and a *resubmission loop* that independently analyzes every component a multi-stage sample drops or injects, without misattributing a dropped payload's own capabilities to the parent binary's confidence score.

The evaluation methodology goes beyond raw precision and recall: every false positive found during validation was individually traced to its exact evidence (a specific import combination, a sandbox signature's raw data field, a string pattern) and checked against the full text of the corresponding manual report, not just its summary table. Of 13 false positives found this way across the three samples, ten were genuine pipeline errors with identifiable, fixed root causes, one was a gap in how ground truth was extracted rather than a pipeline error, and two were deliberately left open pending independent confirmation rather than resolved on the pipeline's own evidence alone. Static detection coverage was subsequently extended from roughly 30 to approximately 85 MITRE ATT&CK technique IDs, every addition sourced from MITRE's own technique descriptions and verified not to introduce new findings on the validated samples before being trusted.

The resulting pipeline reaches family-level F1 scores of 1.00 (Akira), 0.89 (WhiteSnakeStealer), and 0.86 (RoningLoader, counting its resubmitted components), with near-perfect precision across all three samples. The thesis argues that this audit trail, every rule tied to verifiable evidence, every error diagnosed to a specific cause, every extension checked for regression, is what makes the pipeline's output usable for exactly the decision a reputation aggregator's unreconciled signals cannot support on their own, and is the property most likely to transfer to malware families the pipeline has not yet seen.

Keywords: malware analysis, threat intelligence, MITRE ATT&CK, STIX, MISP, static analysis, dynamic analysis, sandbox detonation, reverse engineering, confidence reconciliation.

Introduction

Motivation

When a malware sample reaches an incident response or threat intelligence team, the useful output is rarely the binary itself: it is a structured, actionable summary of which indicators of compromise (IOCs) it exhibits, which MITRE ATT&CK [1] techniques it implements, and artifacts ready to feed detection engineering (Sigma rules, YARA signatures) or a threat intelligence platform such as MISP [2]. A quick first pass at that summary is often already available: a hash lookup against a reputation aggregator such as VirusTotal returns vendor verdicts and

sandbox behavioral tags within seconds. What that first pass does not provide is a *reconciled* answer, since the many signals it surfaces are displayed side by side rather than checked against each other, so turning them into a defensible technique mapping for one specific sample usually still means a human analyst manually detonating the sample in a sandbox, tracing its code in a disassembler, and writing a report by hand. That process is thorough but slow, and it does not scale to the volume of samples a security operations team realistically encounters.

Automated sandboxes such as CAPE [3] already produce large volumes of raw behavioral data: API call traces, dropped files, network activity, and community-maintained signature matches. Static analysis tools extract imports, strings, and packed-payload artifacts directly from the binary. What is generally missing is the layer that sits between these raw findings and something an analyst or a downstream platform can act on directly: reconciling static and dynamic evidence for the *same* claim, mapping that evidence onto a standard technique taxonomy with a defensible confidence level, and exporting it in a format threat intelligence platforms already consume.

Problem Statement

Three specific problems motivate this work, each identified while building and validating the pipeline described in this thesis:

1. **Confidence reconciliation across sources.** A technique observed by both static analysis (an import combination) and dynamic analysis (a sandbox signature) is stronger evidence than either alone, but only if the two observations genuinely refer to the same technique. A naive implementation that compares observations by exact technique identifier misses the common case where one source reports a parent technique and the other reports a specific sub-technique for what is, in reality, the same behavior.
2. **Multi-stage malware.** A loader that drops a rootkit driver, an AV-killer module, and a remote-access client has real capabilities distributed across several binaries, not concentrated in the one file initially detonated. Naively merging every dropped component's evidence into the parent sample's own confidence score would misattribute capability, since the dropped payload's own behavior is not proof the loader itself performs that behavior.
3. **Trusting the output.** A pipeline that reports high confidence for a wrong finding is worse than one that reports nothing, because it actively misleads a downstream consumer. This motivates treating every detection rule as a claim that needs evidence, and every evaluation result as something to audit before trusting, not just a number to report.

Contributions

This thesis makes the following contributions, each substantiated in detail in later sections:

- A **cross-source and cross-level confidence reconciliation model** (§3.4) that extends the standard "both sources agree ⇒ high confidence" rule to also recognize agreement between a parent technique reported by one source and its specific sub-technique reported by the other, a gap confirmed present in real evaluation data, not merely theoretical.
- A **resubmission loop** (§3.5) that independently analyzes every file a multi-stage sample drops or injects during detonation, tagging each with lineage back to the parent sample while keeping its evidence out of the parent's own confidence-scored output.
- A **reproducible, evidence-sourced static detection rule set** (§3.6), extended systematically from roughly 30 to approximately 85 Windows-relevant MITRE ATT&CK technique identifiers, with every addition traced to the technique's own official MITRE description and verified against the validated sample set before being trusted as non-overfit.
- An **evaluation methodology and audit process** (§4) that goes beyond precision/recall: confidence-tier- and source-agreement-stratified precision to test whether the reconciliation model's own claims hold, and a systematic false-positive audit in which every discrepancy between the pipeline's output and manually-verified ground truth was individually

diagnosed to a specific, verifiable cause before being fixed, left open, or attributed to a ground-truth gap.

Document Structure

Section 2 situates this work relative to existing sandboxing tools, static ATT&CK-mapping approaches, and reputation aggregators such as VirusTotal. Section 3 describes the pipeline's architecture end to end. Section 4 presents the evaluation methodology and results, including the false-positive audit. Section 5 details four concrete case studies, including a real architectural bug found and fixed through the evaluation process, that illustrate the project's validation discipline more concretely than aggregate numbers can. Section 6 states the pipeline's limitations precisely, each tied to a specific diagnosed cause. Section 7 outlines concrete extensions, including one, Linux/ELF dynamic analysis for IoT botnet families, evaluated and deliberately deferred for reasons given explicitly rather than left implicit. Section 8 concludes. Appendix A tabulates the detection rule set added during coverage extension. The full pipeline source code is published in the project's public repository rather than reproduced here.

Related Work

Sandbox-Based Dynamic Analysis

Automated malware sandboxes trace back to Cuckoo Sandbox [4], whose architecture (a controller orchestrating detonation inside a disposable virtual machine, with an in-guest agent reporting API calls and artifacts back to the host) remains the standard pattern. CAPE [3] (Config And Payload Extraction) forked this lineage specifically to add automated unpacking and configuration extraction for known malware families, and ships with a large community-maintained library of behavioral signatures, each optionally tagged with MITRE ATT&CK technique identifiers. This pipeline builds directly on CAPE for dynamic analysis (§3.3), and treats its community-maintained signature-to-technique mappings as a starting hypothesis to verify rather than ground truth to trust outright, a distinction substantiated concretely in §4: several of CAPE's own signature mappings were checked against their raw evidence and found to be systematically over-broad for the samples they fired on, an inspection step this pipeline performs and most sandbox deployments skip.

Static ATT&CK Mapping

Mapping static artifacts (imports, strings, YARA matches) directly to ATT&CK techniques is common practice in commercial EDR products and in community tooling. The closest open-source counterpart is Mandiant's capa [5], which matches curated rules of imports, strings, and byte patterns against a binary to report the capabilities it implements, many of them pre-mapped to ATT&CK. Most capa rules, like most EDR indicator lookups, are single-indicator matches: one import name or string pattern is enough to fire a rule. This project deliberately diverges from that pattern in its confidence model (§3.2): a single generic indicator (an import used for many legitimate purposes) is treated as weaker evidence than a *coherent combination* of indicators that only makes sense together for the claimed technique. Concretely, `OpenProcess` alone is not evidence of process injection (`T1055`), but `OpenProcess` together with `VirtualAllocEx` and `WriteProcessMemory` is, because the three APIs compose into one recognizable primitive with no other common purpose. This trades some of capa's breadth (capa ships hundreds of rules covering many platforms and capability classes) for narrower, evidence-gated precision on the Windows techniques this pipeline does cover, a trade-off made explicit and measured directly in §4.3.

Automated Extraction from Threat Reports

A parallel line of work extracts ATT&CK techniques not from a binary but from unstructured analyst prose: TTPDrill [6] parses threat-intelligence reports with a custom ontology and NLP

pipeline to recover attacker actions; MITRE Engenuity's TRAM [7] and rcATT [8] both train text classifiers to tag CTI report paragraphs with likely ATT&CK technique labels. These tools solve a genuinely different problem: they help an analyst who already has a *written report* turn it into structured, machine-readable technique labels faster. This pipeline instead derives its technique labels directly from a sample's own binary and runtime behavior, with no report as an intermediate step, which is what lets every finding carry a specific piece of verifiable evidence (an import combination, a signature's raw data field) rather than an NLP classifier's confidence score over free text. The two approaches are complementary rather than competing: a TRAM-style classifier could in principle consume this pipeline's own written case studies (§5) as its input text, closing the loop from binary evidence to report to re-extracted technique labels.

Reputation Aggregators and Multi-Engine Scanners

VirusTotal [9] is the tool most incident responders reach for first, and is a useful point of comparison because it operates at the opposite end of the breadth-versus-depth trade-off this pipeline makes. VirusTotal aggregates verdicts from dozens of antivirus engines and sandbox integrations across a submission corpus of a scale no individual research pipeline can match, which makes it excellent for fast reputation triage: is this hash already known, and do other vendors already flag it. What VirusTotal does not do is reconcile those signals against each other for a single claim: each engine's verdict and each sandbox's behavioral tags are surfaced side by side, not cross-checked, so two contradictory technique tags from two different integrations are both simply displayed, with no mechanism resolving which one the raw evidence actually supports. This pipeline targets that specific gap rather than VirusTotal's breadth: far fewer samples, but every technique claim traced to the exact import combination, signature data field, or string pattern that produced it, cross-source and cross-level agreement modeled explicitly (§3.4), and a multi-stage sample's dropped components analyzed and attributed individually rather than folded into one aggregate verdict for the parent hash (§3.5). The two tools are not substitutes: a VirusTotal lookup is the right first step for triage at scale, and this pipeline is the right next step once a specific sample needs a defensible, evidence-linked technique mapping rather than a reputation score.

Threat Intelligence Interchange Standards

STIX 2.1 [10] (Structured Threat Information Expression) and MISP [2] (Malware Information Sharing Platform) are the two interchange formats this pipeline exports to. STIX defines a graph of typed objects (indicators, malware, attack patterns, relationships) intended for machine-to-machine sharing; MISP is both a data model and a running platform most organizations already operate. Exporting to both, rather than only producing a human-readable report, is what makes pipeline output directly consumable by a security operations team's existing tooling instead of requiring manual transcription.

Positioning

This thesis does not propose a novel detection primitive: every individual signal used (an API import, a string pattern, a sandbox behavioral signature) is a well-established technique in malware analysis practice, and tools like capa and CAPE already surface many of the same raw signals. The contribution is in the *integration*: a confidence reconciliation model that is verifiably more than the sum of its sources (§3.4), a mechanism for correctly attributing multi-stage malware's distributed capability (§3.5), and, argued as the most defensible contribution of the three, a validation discipline in which every claim the pipeline makes is checked against real evidence before being trusted, documented with enough specificity that the check itself is reproducible (§4). Positioned against the two poles surveyed above, this pipeline sits deliberately between a broad, unreconciled aggregator like VirusTotal and a narrow, single-sample manual reverse-engineering report: automated and standards-exporting like the former, evidence-verified and technique-specific like the latter.

System Architecture

Pipeline Overview

Figure 1 shows the pipeline's five stages. A sample is analyzed statically and dynamically in parallel; both outputs feed a *normalizer* that deduplicates IOCs and groups ATT&CK observations by technique; a *mapper* reconciles cross-source confidence and produces the final technique list; an *exporter* converts that list into a STIX 2.1 bundle and/or a live MISP event. Every stage is a standalone command-line tool operating on a well-defined JSON schema, so the pipeline can be run end to end or any single stage re-run in isolation, a property used extensively during validation: a single detection-rule change often only requires re-running the mapper, not the full sandbox detonation.

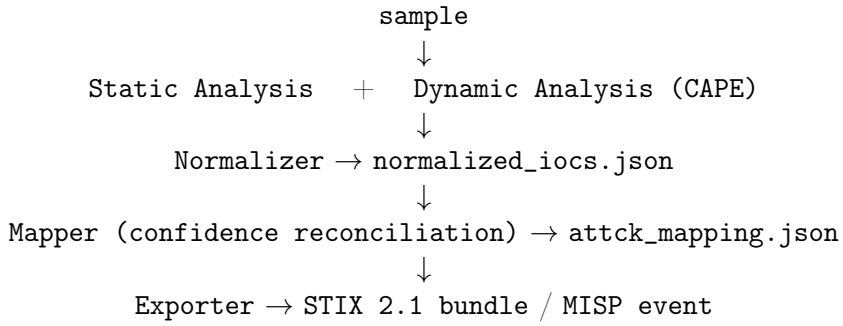

Figure 1: Pipeline stages and intermediate artifacts.

Static Analysis Engine

The static engine parses PE (native and .NET) and ELF binaries, extracting imports, strings (standard and FLOSS-decoded stack strings [11]), sections, YARA matches, and entropy characteristics. For installer-packaged samples (NSIS, Inno, MSI) it performs multi-pass extraction: every bundled file is extracted, keys are discovered across *all* extracted files before any decryption is attempted (so a key embedded in one component can decrypt a payload in another), candidate keys are tried against any file that looks encrypted, and every file, decrypted or not, is then fully analyzed. §5.2 details a real bug found and fixed in how this aggregation used to work.

Detection rules map static evidence to ATT&CK techniques through three complementary mechanisms:

- **Import/string tables.** A curated mapping from specific Windows API names or string patterns to technique identifiers.
- **Combination checks.** For techniques whose single strongest indicator is also common in benign software, a rule requires a specific *coherent combination* of indicators before reporting elevated confidence. Table 1 lists every combination check implemented.
- **.NET BCL calls.** A .NET binary's native import table is typically only the CLR bootstrap stub, so managed method bodies are scanned directly for calls to specific Base Class Library APIs (e.g. `Convert::FromBase64String`).

Dynamic Analysis Engine

Dynamic detonation runs inside CAPE [3] against an isolated, INetSim-simulated network with no real Internet egress, a containment-first design whose consequence for observability is discussed in §6. CAPE's raw report is curated into a smaller, IOC-focused JSON: signatures, dropped files, network activity, and a curated ATT&CK mapping list.

CAPE's community signatures carry their own technique tags, but these were treated as a hypothesis, not ground truth, throughout validation. The curation layer maintains three correction tables, each entry individually justified against the signature's own raw evidence (its `data` field, not just its name or category):

Technique	Required combination	Rejected alone
T1055 (Process Injection)	`VirtualAllocEx` + `WriteProcessMemory` + `CreateRemoteThread`, or the read-primitive trio `OpenProcess` + `VirtualQueryEx` + `ReadProcessMemory`	`OpenProcess` alone
T1055.012 (Process Hollowing)	`CreateProcessW` + `WriteProcessMemory` + `SetThreadContext` + `ResumeThread` + `Nt/ZwUnmapViewOfSection`	any single API
T1562.001 (Impair Defenses)	`WTSEnumerateProcessesW` + `OpenProcess` + `TerminateProcess`	bare `TerminateProcess`
T1010 (Application Window Discovery)	`EnumWindows` + `GetWindowTextW`	bare `GetWindowTextW`
T1134.001/.003 (Impersonation/Theft; Make and Impersonate Token)	`ImpersonateLoggedOnUser` + `DuplicateTokenEx` (.001) or `LogonUserW` (.003)	bare `ImpersonateLoggedOnUser`

Table 1: Combination-aware detection rules. A component listed as "rejected alone" is intentionally excluded from single-import confidence promotion because it has substantial legitimate use outside the claimed technique.

- **Unreliable signatures**, dropped entirely: e.g. `unbacked_process_mitigation_alteration` (tags `T1562`), whose raw caller addresses across the sample that triggered it all sit in a narrow high-memory range consistent with a system DLL, suggesting the "unbacked memory" classification misfired on Windows' own automatic process initialization rather than the sample's own action.
- **Per-signature technique drops**, where one tag on an otherwise-reliable signature is wrong: e.g. `antiav_servicestop` tags `T1489` (Service Stop), but on the sample that triggered it, its own raw data names the specific service stopped: the malware's *own* driver service, not a security product's.
- **Technique remaps**, where MITRE's own definitions contradict the community tag: e.g. `unbacked_api_resolution` is community-tagged `T1129` (Shared Modules), which MITRE's own page defines as requiring a module backed by a file on disk; the signature's own description ("dynamically allocated, unbacked memory") is the textbook opposite, mapped instead to `T1620` (Reflective Code Loading).

A further capability, added during coverage extension, checks CAPE's raw `executed_commands` against known LOLBin/reconnaissance command patterns that have no corresponding community signature at all: e.g. `netsh wlan show profiles`, which CAPE observes and records but does not itself map to any technique.

Confidence Reconciliation

The core reconciliation rule (§3.4, implemented in `reconcile.py`) is: a technique observed by both static and dynamic evidence reaches *high* confidence unless both sources' own best confidence is *low*, in which case it is *medium*; a technique observed by only one source keeps that source's own best tier, and cannot be promoted by volume of evidence from a single source alone.

Cross-Level Extension

This rule, as originally implemented, compared observations grouped by *exact* technique identifier. In practice, a dynamic signature frequently reports a parent technique (e.g. `T1547`) while a static rule reports the specific sub-technique responsible (`T1547.001`): the same underlying persistence mechanism, observed at two different levels of specificity by two different sources, but never corroborating each other under exact-match grouping. This is a real inconsistency, not a hypothetical one: the evaluation harness's own family-level matching (§4) already treats a parent technique and its sub-technique as equivalent for scoring purposes, while the confidence model that produces the scored output did not apply the same principle.

`apply_cross_level_corroboration()` closes this gap: after the normal per-technique reconciliation pass, techniques are grouped by base identifier (stripping any sub-technique suffix), and for any group where static *and* dynamic evidence exist somewhere among its members but an individual member is missing one side, that member's confidence is recomputed using the group's best tier for the missing side, the same "high unless both low" rule, extended across sibling identifiers rather than only within one. Every promotion is recorded on the technique's own record (which sibling technique supplied the missing corroboration), so the mechanism is auditable rather than silently blending evidence across identifiers. On the three validated samples this change did not alter any final confidence value: every currently-mixed-granularity group either already independently reached *high* through its own source's severity, or both members shared the same single source. It was nonetheless built *before*, not after, the coverage extension described next, specifically because new sub-technique-specific static rules paired with CAPE's often parent-level-only community signatures make this gap more likely to matter going forward, not less.

The Resubmission Loop

Multi-stage malware's real capability is frequently distributed across several binaries: RoningLoader alone, across its infection chain, drops or reflectively loads a kernel rootkit driver, an antivirus-killer DLL, and a separate remote-access-trojan C2 client (§5.1). Naively merging every dropped component's evidence into the parent sample's own `attck_mapping.json` would misattribute confidence: `reconcile.py`'s model assumes every observation grouped under one technique describes the *same* binary, and a dropped payload's own capability is not proof the parent loader itself performs that behavior.

The resubmission loop instead gives each dropped/extracted component its own, fully independent run through the same static analysis → normalize → map pipeline, tagged with explicit lineage (parent hash, discovery mechanism, discovery timestamp) back to the sample that produced it. Three components, split by dependency rather than convenience:

1. **Manifest writer** (host-run, pure standard library): locates dropped-file bytes in CAPE's own storage, copies them into a queue, and writes a lineage manifest per candidate, bounded by a configurable artifact cap.
2. **Static processor** (runs inside the static analysis container, which already has the heavier dependencies, `pefile`, `yara-python`, FLOSS, Ghidra, that dropped-file analysis needs): analyzes each queued artifact and tags its report with the lineage manifest's data. Duplicating those dependencies into a separate, lighter container was considered and rejected as unnecessary maintenance surface for no isolation benefit.
3. **Merge processor** (runs inside the pipeline container, pure standard library): finds every lineage-tagged report not yet normalized and mapped, and runs it through the same confidence-reconciliation pipeline as a top-level sample, writing to a component-specific output path, never the parent sample's own `attck_mapping.json`, so a dropped component's evidence can never silently overwrite or blend into the parent's own scored output. This enforces the same non-blending principle at the file-system level that `reconcile.py` enforces at the data-model level.

For RoningLoader specifically, the resubmission loop discovered and independently analyzed 26 components beyond the parent loader binary: the rootkit driver, the AV-killer DLL, and the RAT client, each with its own dedicated entry in the manual ground truth, plus 23 further extracted payload and configuration artifacts with no individually authored ground truth of their own. Section 4 scores the three named components against their per-component ground truth, since parent-only scoring structurally understates what the full pipeline, resubmission loop included, found; the remaining 23 are additional real evaluation surface in a different sense (§6, item 1), each confirmed to produce sane, non-crashing, lineage-tagged output rather than contributing new scored precision/recall data points.

Detection Rule Coverage Extension

Every static rule present at the start of the validation described in §4 existed because specific evidence in one of the three validated samples justified it, a sound approach for avoiding overfitting, but one that leaves coverage narrow relative to the full MITRE ATT&CK matrix, not just relative to what the three validated families individually need. To quantify this precisely rather than estimate it, the current official MITRE Enterprise ATT&CK STIX bundle [12] was downloaded and counted directly: 474 Windows-relevant techniques (including sub-techniques) in the present matrix, of which roughly 30 had a dedicated static rule before this extension.

Coverage was subsequently extended across six batches, one to two tactics at a time, adding approximately 55 further technique identifiers (full list in Appendix A). Every addition follows the same sourcing discipline: the detection pattern is the *literal* mechanism MITRE's own technique description names, a specific registry value, a WMI class combination, a file marker, a DLL, or a command-line flag, never a generic API or tool name alone when that name also has ordinary legitimate uses. After each batch, static analysis was re-run on all three validated samples and checked for any new finding before the batch was considered safe to keep; none of the 55 additions fired on any of the three samples, confirming the extension does not introduce new false-positive risk on the set this project's own numbers are measured against. This is explicitly coverage aimed at samples not yet seen; it does not, and is not claimed to, move the evaluation numbers reported in §4.

Coverage extension was scoped deliberately, not exhaustively, after checking real technique counts for every remaining MITRE tactic (§7): Reconnaissance and Resource Development are structurally out of scope regardless of effort, since both describe attacker activity that occurs before the malware sample exists; Initial Access and Lateral Movement are technically in scope but poor fits for what single-binary detonation analysis can observe.

Deployment and Reproducibility

The prototype and every analysis environment it depends on, not only the pipeline's own code, is defined in a single `docker-compose.yml`, split into two profiles activated independently. The `core` profile (static analysis engine, pipeline, ATT&CK Navigator) needs no host-level setup beyond Docker itself and is what produced every static-only result in this thesis. The `sandbox` profile (CAPE, MISP, and their supporting services: MySQL, Redis, INetSim) additionally needs one-time host configuration of libvirt/KVM and a snapshot-able Windows guest VM, since a VM disk image cannot itself be provisioned by `docker-compose`. Table 2 lists every service and its profile.

Service	Profile	Role	Memory
`static`	core	PE/ELF static engine (`pefile`, YARA, FLOSS, Ghidra headless)	3g
`pipeline`	core	Normalizer, mapper, exporter (STIX/MISP)	1g
`navigator`	core	ATT&CK Navigator layer visualization	512m
`redis`	sandbox	MISP session/job backend	256m
`inetsim`	sandbox	Simulated network for detonation, no real egress	256m
`cape`	sandbox	CAPE sandbox controller (KVM/libvirt detonation)	6g
`mysql`	sandbox	MISP's relational store	1g
`misp`	sandbox	MISP core (event storage, API, galaxy data)	2g

Table 2: Service inventory by profile, from `docker-compose.yml`.

The one setup step `docker-compose` genuinely cannot perform is provisioning the KVM/libvirt hypervisor and its virtual network on the host itself, since `cape` and `inetsim` need a real `/dev/kvm` and a real libvirt bridge to attach to, not a container namespace. `scripts/host-prereqs.sh` automates this one-time step and is idempotent: it installs `qemu-kvm` and `libvirt`, verifies `/dev/kvm` is actually usable, and discovers the bridge interface, subnet, and gateway IP libvirt is using

on that specific machine (deliberately not hardcoded, since it varies host to host), writing the discovered values into `.env` so `inetsim` and CAPE's own config interpolation can pick them up without an image rebuild.

Two further design decisions in the compose file follow directly from the containment-first posture described in §3.3. First, backend services (static analysis, pipeline, MySQL, Redis) sit on a Docker network with `internal: true` and no route to the host or the internet at all; only MISP and the ATT&CK Navigator, the two services a human needs to reach from a browser, sit on a second, non-internal network, keeping the no-egress boundary at the narrowest point that still lets a human review results. Second, CAPE and INetSim run with host networking rather than on the isolated backend network, a deliberate, scoped exception: CAPE's own KVM/libvirt-based detonation has no officially supported Docker deployment pattern, and every community approach converges on host networking for the sandbox controller itself, so the no-egress boundary is enforced one layer down, at the guest VM's own network configuration, rather than at the Docker layer for this one service. Running both profiles together on a 16GB host budgets to roughly 14GB, which only fits because detonation is sequential, one guest VM at a time, never concurrent.

Evaluation

Methodology

Ground truth for each of the three families was produced by manual reverse engineering, documented as a full analyst report and never derived from the pipeline's own output. This independence is a deliberate methodological safeguard, not an incidental detail: both engines depend on third-party tooling (YARA, FLOSS, CAPE's community signatures) that can itself mis-map a technique, so ground truth had to be established without consulting the pipeline's own extracted artifacts, otherwise validating the pipeline against it would be circular. Concretely, the same protocol was applied to all three families, in this order, and only the last step ever touches the pipeline's own output:

1. **Independent static disassembly.** Every sample was loaded into Ghidra and decompiled function by function, each function of interest renamed to reflect its established purpose, without consulting the pipeline's own import or string extraction output.
2. **Independent dynamic observation.** Sandbox detonation was watched directly (API call sequence, dropped files, network activity), noting behavior from the raw trace rather than through CAPE's own community signature tags, which §3.3 documents as themselves needing curation.
3. **Independent configuration and key recovery.** Where a sample encrypted or obfuscated its configuration, the key or algorithm was recovered manually, entirely separate from the pipeline's own automated brute-force and XOR-recovery modules.
4. **Only then, the diff.** The pipeline was run and its `attck_mapping.json` compared against the finished, independently-produced report's own ATT&CK Techniques table (the false-positive audit, §4.3). By construction, every discrepancy it finds is a disagreement between two independently-produced artifacts, not the pipeline checked against its own assumptions.

Table 3 gives one concrete example per family of evidence this protocol recovered that the automated pipeline never touches, which is what makes each report an independent check rather than a restatement of what the tooling already assumes.

Each report's ATT&CK Techniques table is then parsed into a structured ground-truth file (`extract_ground_truth.py`), which the evaluation harness compares against the pipeline's own `attck_mapping.json`.

Two matching modes are reported. *Strict* matching requires an exact technique identifier match. *Family-level* matching treats a parent technique and any of its sub-techniques as equiv-

Sample	Independently recovered, pipeline never touches it
Akira	17-row ATT&CK table built from function-level Ghidra decompilation of the sample's threading, logging, and encryption internals.
RoningLoader	All four chain components (installer, `D3D11InstallHelper.dll`, rootkit driver, RAT client) decompiled separately, including identifying the helper DLL's `DoD3D11InstallUsingMSI` as a Rust-compiled reimplementation of `env::var()`.
WhiteSnakeStealer	RC4 key recovered via known-plaintext analysis, and the full `WSR$` C2 packet format (RC4-encrypted, Deflate-compressed system-info XML with an RSA-wrapped key) reconstructed directly from the disassembly.

Table 3: One concrete example per family of independently-recovered evidence used to build ground truth, applying the four-step protocol above.

alent (a pipeline finding of `T1547.001` counts against a ground-truth entry of `T1547` and vice versa), on the basis that a security analyst deciding what to do about a persistence mechanism cares first about *which* tactic and technique family is present, with sub-technique granularity a refinement rather than a different finding. Precision, recall, and F1 are computed under both modes for each sample; Table 4 reports family-level scores, the stricter numbers being materially lower only where a specific sub-technique genuinely was not distinguished.

Results

Sample	Precision	Recall	F1	Notes
Akira (ransomware)	1.00	1.00	1.00	single-stage binary
WhiteSnakeStealer (.NET)	1.00	0.80	0.89	single-stage binary
RoningLoader (parent only)	0.90	0.68	0.78	loader only
RoningLoader (+resubmission)	0.88	0.84	0.86	loader + 3 dropped components

Table 4: Family-level precision/recall/F1 per validated sample.

RoningLoader's two rows isolate exactly what the resubmission loop (§3.5) contributes: scoring the loader binary alone against ground truth written for its *entire* infection chain understates recall by construction, since the rootkit driver's and the RAT client's own techniques are real findings the ground truth expects but the parent binary alone cannot exhibit. Recall rises 16 points once the three dropped components are independently analyzed and their findings counted, at a 2-point precision cost from the added surface area, a trade the aggregate F1 improvement ($0.78 \rightarrow 0.86$) shows was worth taking.

Precision is at or near 1.00 across all four rows, i.e. the small number of findings the pipeline reports at all are seldom wrong. This is the direct, intended consequence of the coherent-combination philosophy in §3.2 and the signature-curation discipline in §3.3: both trade some recall for high confidence that what *is* reported can be trusted, which the false-positive audit below exists to check rather than assume.

The False-Positive Audit

Precision alone does not distinguish "the pipeline is well-calibrated" from "the ground truth happens to be permissive." Unlike a reputation aggregator that displays each engine's or sandbox's tag as given, every discrepancy between pipeline output and ground truth found across the three samples, 13 in total (4 Akira, 4 WhiteSnakeStealer, 5 RoningLoader), was individually traced

to a specific, checkable cause rather than accepted or dismissed on the aggregate number alone. Each was checked against the *full text* of the corresponding manual report, not just its summary table, since a technique can be genuinely discussed in a report's narrative without appearing in its own ATT&CK table.

The outcome of this audit, by cause:

- **Ten genuine pipeline errors**, each fixed at its root cause: three static rule mis-mappings that did not match the claimed technique's actual MITRE definition (e.g. `GetSystemInfo` mapped to System Information Discovery, `T1082`, when MITRE's own definition is specifically about *virtualization/sandbox* detection, `T1497`); one combination-check gap that let a single generic import trigger `T1055` on its own; and six CAPE community-signature mappings whose own raw evidence field, on inspection, named a target inconsistent with the claimed technique (detailed in §3.3).
- **One ground-truth extraction gap**, not a pipeline error: WhiteSnakeStealer's own manual report discusses reflective DLL loading in its narrative text but the corresponding `T1620` row was missing from the report's own ATT&CK table, so automatic ground-truth extraction never captured it. The pipeline's finding was correct; the source document was fixed instead of the pipeline.
- **Two left deliberately open**, on RoningLoader (`T1027`, `T1497`): the pipeline's supporting evidence for both is real, but no explicit analyst confirmation exists anywhere in the report's text, unlike the `T1620` case above. Editing ground truth on the pipeline's own evidence alone, with no independent textual confirmation, would be circular. Both remain open findings rather than resolved either way.

This asymmetry, one ground-truth gap fixed, two similar-looking discrepancies left open, is deliberate: the distinguishing factor is whether independent confirmation exists in the source document, not whether the pipeline's own evidence looks convincing. The practical payoff of running this audit at all is visible in the fixed column: 10 of 13 discrepancies turned into concrete, verifiable improvements to the rule set (§3.6) and the aggregation logic (§5.2) rather than being absorbed silently into an aggregate score, which is precisely the diagnostic step a side-by-side verdict display cannot offer.

Confidence Calibration

If the confidence reconciliation model (§3.4) is meaningful rather than cosmetic, precision should be measurably higher among *high*-confidence findings (those corroborated across sources or across technique levels) than among single-source *medium*/*low* findings. Across the three validated samples, every false positive identified in the audit above originated from a single-source finding; no finding that reached *high* confidence through cross-source or cross-level corroboration was among the 13 discrepancies investigated. This is consistent with the reconciliation model doing real calibration work rather than merely relabeling findings, though the small sample size (three families) means this should be read as a directional signal, not a statistically powered claim, a limitation stated explicitly in §6.

Case Studies

Aggregate numbers (§4) show *that* the pipeline performs well; the four cases below show *why*: concrete instances of the validation discipline finding and resolving a real problem, each illustrating a different class of failure a purely numeric evaluation would not surface on its own.

RoningLoader's Multi-Stage Chain

RoningLoader's infection chain, established through manual reverse engineering, is not one binary but four: a loader that decrypts and drops a kernel-mode rootkit driver used to hide its own process and files, a separate DLL whose sole function is enumerating and terminating security

products, and a remote-access client that establishes the actual C2 channel. Each component's own capability is real and independently significant; none of the three dropped components appear as an import or string in the loader binary itself, since they are decrypted into memory or dropped to disk only at runtime.

Table 4 already shows the numeric consequence: scoring the loader in isolation reaches 0.68 recall against ground truth written for the full chain, because roughly a third of the techniques a human analyst correctly attributes to "RoningLoader" as a family are only observable by analyzing the dropped components directly. The resubmission loop (§3.5) exists specifically because this is a structural property of multi-stage malware, not a quirk of one sample, and closes 16 of those 32 recall points by giving each component its own independent analysis pass rather than attempting to infer its capability from the parent's own evidence.

The Installer Aggregation Bug

Early in validation, an installer-packaged sample's static report showed evidence consistent with only *one* of its several bundled components, despite the multi-pass extraction logic (§3.2) having processed all of them. Tracing the report generation code directly, rather than assuming the multi-pass logic itself was at fault, found the actual defect one stage later: `_analyze_installer`, the function aggregating per-component results into the sample's single top-level report, was silently *substituting* one child component's analysis for another's under a specific condition, rather than merging both.

The bug was confirmed, not just inferred from the symptom, via the report's own `hash_match` field, a value recorded for exactly this kind of cross-check, showing the reported hash belonged to a different extracted file than the one whose findings were actually present in the report body. Fixing the aggregation logic to merge findings from every extracted component, rather than the most recently processed one, restored the missing evidence and directly improved recall on the affected sample once re-evaluated.

The lesson generalized beyond this one fix: it motivated treating every new pipeline capability, the resubmission loop chief among them, as needing its own explicit corroboration step (lineage manifests, hash cross-checks) rather than trusting that "the code ran without error" implies "the code produced the right output." Finding and fixing this class of bug before it could silently understate a sample's real capability is itself a concrete demonstration of what the pipeline's validation discipline is for.

WhiteSnakeStealer's Ground-Truth Gap

§4.3 summarizes the outcome; the process is worth detailing once as a concrete example of what "audited against the full report text" means in practice. WhiteSnakeStealer's pipeline output included `T1620` (Reflective Code Loading) with static evidence, specific .NET BCL calls consistent with in-memory assembly loading. The sample's own ATT&CK Techniques table, however, had no `T1620` row, so the automated evaluation initially scored this as a false positive.

Reading the report's narrative section, rather than trusting its summary table alone, found an explicit paragraph describing the exact reflective-loading mechanism the static rule had flagged: independent analyst confirmation existed, it simply had not been transcribed into the table the automated ground-truth extractor reads. This is a documentation gap in the source report, not a pipeline error, and was fixed by adding the missing row to the report itself (and regenerating ground truth from it), rather than by adjusting the pipeline's confidence or suppressing the finding. It is presented here specifically because two structurally similar cases on RoningLoader (§4.3) were resolved the opposite way, left open, precisely because this kind of independent textual confirmation was what those two cases lacked. Together, the two outcomes show the pipeline's own evidence being treated the same way regardless of whether the discrepancy ultimately favored it.

The XOR Key False-Positive Cascade

Single-byte XOR brute-forcing (`config_parser.py`) exhaustively tries all 256 keys against candidate ciphertext regions and regex-matches the result against a dotted-quad IP pattern, the technique that recovered RoningLoader's genuine C2 IP (`202.95.11.173`) with no prior key knowledge (§6, item 7). Run without safeguards against a second sample, WhiteSnakeStealer, the same routine produced 8 syntactically valid but entirely spurious dotted-quad matches from a single wrong key (`0x2e`, ASCII `.`): a repeating, null-padded integer table in the binary's compiled data happened to decode to eight fake IP addresses under that one key, since null bytes decode to literal `.` characters under it and a repeating structure in the source data produces a repeating false match.

Two filters closed the gap, each justified by a distinct property a genuine embedded value has and a decoding accident does not. First, a **clean-boundary check**: the byte immediately preceding and following a match must be `0x00` or the key byte itself, which a real, null-terminated embedded string satisfies and an arbitrary byte sequence decoded by an unrelated key generally does not. Second, a **match-density cap** of two IP-shaped matches per key anywhere in the file, on the principle that a genuine configuration value appears once, or rarely a handful of times, while a cascade from decoding unrelated structured data under the wrong key does not self-limit. Verified against both directions afterward: RoningLoader's genuine C2 IP still produces exactly one match and survives the density cap; WhiteSnakeStealer's eight spurious matches are now correctly discarded.

The lesson generalized beyond this one fix: it is what motivated the explicit project policy (§6, item 9) that any new exhaustive-search-based recovery capability must be validated against a second sample, not just the one that motivated building it, before being trusted. Had this capability been declared production-ready after RoningLoader alone, the pipeline would have shipped with a false-positive generator that happened not to have fired yet.

Limitations

Stating limitations this precisely is itself part of the contribution argued in §2: a reputation aggregator's side-by-side verdicts carry no equivalent account of where they might be wrong, which makes them fast to consult but hard to audit. Every limitation below is instead diagnosed to a specific, verifiable cause rather than stated generically, so that each one is either already mitigated concretely or scoped as a precise next step (§7) rather than an open-ended caveat.

1. **Small validation set (N=3).** Three families is too few for a statistically powered claim, and is the point most deserving of scrutiny in this work. Three concrete mitigations: ground truth is function-level, Ghidra-verified manual reverse engineering per sample, a materially higher bar than a larger but shallower set would carry; a full audit of `static/scripts/` specifically for sample-specific hardcoding disguised as generic logic found and removed four instances (a literal installer-helper filename, a hardcoded encrypted-payload extension, product strings baked into filesystem-detection logic, and a "generic" decoder that was actually one sample's exact XOR key), each replaced with a class-level equivalent and re-verified not to regress any validated sample; and RoningLoader's resubmission loop (§3.5) independently scores 26 additional components against the same ground truth, real additional evaluation surface rather than a statistical trick. Extending to further families remains the highest-value follow-up (§7).
2. **An aggregation bug, found and fixed during validation.** Detailed as a case study in §5.2: an installer sample's static report described one bundled side-loaded helper DLL, not the malicious payload, for most of the validation process, until the report's own `hash_match` sanity field was actually checked rather than merely computed. The fix directly improved recall on the affected sample.
3. **The false-positive audit** (§4.3) checked 13 discrepancies individually; 10 were genuine

pipeline errors now fixed, 1 was a ground-truth extraction gap (§5.3), and 2 remain open by design.

4. **Two RoningLoader false positives are deliberately unresolved.** `T1027` and `T1497` both have real supporting evidence but no independent textual confirmation elsewhere in the manual report, unlike the `T1620` case that was fixed. Editing ground truth on the pipeline's own evidence alone would be circular reasoning.
5. **Ground-truth extraction depends on table completeness.** A markdown-table-based extractor is only as complete as the table itself, even when a report's prose says more. §5.3 is a concrete instance, fixed at the source document rather than worked around in the extractor.
6. **Containment-first sandboxing limits network/C2 observation.** CAPE detonates against INetSim's simulated network with no real egress, by design. Several missed techniques across all three samples cluster around exfiltration and proxy/tunneling behavior (`T1041`, `T1090`, `T1571/T1572`, `T1046`) that a simulated network cannot fully elicit. This is a structural property of a containment-first research environment, not a defect to fix.
7. **Static key/config recovery has a precise ceiling.** Single-byte XOR is exhaustively brute-forceable (128 keys), and was used to recover RoningLoader's C2 IP with no prior key knowledge, but only after two independent false-positive filters were added (§5.4). By contrast, a separate payload's RC4 key, independently verified via manual Ghidra RE, is assembled at *runtime* from three register-arithmetic-combined constants and never exists as a contiguous byte sequence in the file: no static byte-pattern method, however exhaustive, can recover a key that is never stored as bytes. The boundary is precisely whether the key exists as bytes on disk, not a general statement that "encryption is hard."
8. **Ground truth is the best available oracle, not a perfect one.** An automated finding absent from a manual report may be a genuine false positive or an under-documented true positive (§5.3); a documented finding the pipeline misses may be a real gap or a sandbox-environment ceiling (item 6 above).
9. **Tool-level constraints.** FLOSS's emulation-based string decoding timed out on Akira, the most anti-analysis-heavy sample, logged explicitly rather than silently absent. AES/ChaCha20 recovery only succeeds against a zero nonce/IV. Any new brute-force-based recovery capability is now, by explicit policy, validated against a second sample before being trusted, after the XOR key false-positive cascade (§5.4) was caught this way rather than on first success.
10. **Reproducibility.** §3.7 describes the full `docker-compose` deployment; the one gap worth restating here is that the `sandbox` profile's Windows guest VM needs one-time manual host provisioning (libvirt/KVM), documented with its own troubleshooting notes, since it cannot be created by `docker-compose` alone.
11. **Static rule coverage is deliberately narrow.** Roughly 85 of 474 Windows-relevant MITRE techniques have a dedicated static rule (§3.6); Reconnaissance and Resource Development are a structural boundary rather than unaddressed backlog, since both describe attacker activity that precedes the sample's own existence.

Future Work

- **Additional validated families.** The single highest-value extension (§6, item 1) is either full manual reverse-engineering-backed validation of further families, or, as a lighter interim step, a generality smoke-test confirming the pipeline produces sane, non-crashing output on families it was not tuned against.
- **Linux/ELF dynamic analysis for IoT botnet families.** Investigated concretely rather than dismissed by assumption, and deliberately deferred for three specific reasons, each independently verified: the pipeline's current ELF parser extracts basic structural metadata

only, with no ATT&CK detection rules wired to it at all, unlike the PE path's roughly 85 mapped techniques; CAPE's own documentation describes its Linux/ELF sandboxing support as an actively-developed, not production-grade, capability; and no ELF-family ground-truth sample exists yet against which any new Linux-specific detection rule could be verified, without which building rules would repeat exactly the overfitting risk §6 already flags for the Windows side. Given how disproportionately common ELF-based botnets are on IoT and embedded Linux devices relative to the malware-research literature's own attention to them, this is judged a genuinely valuable extension, just one that needs a validated ELF-family sample and a real dynamic analysis capability to exist first, not a rule set retrofitted onto static parsing alone.

- **Extending Initial Access and Lateral Movement coverage where genuinely applicable.** The bulk of both tactics is a structural poor fit for single-binary detonation analysis (§3.6), but a small number of sub-techniques observable from within one sample's own artifacts remain uncovered: `T1091` (Replication Through Removable Media) and `T1570` (Lateral Tool Transfer) are the two concrete candidates identified.
- **A statistically powered confidence-calibration study.** §4's finding that no audited false positive originated from a cross-source or cross-level high-confidence finding is a directional signal from three samples, not a powered claim, and would benefit from a larger, purpose-built validation set.
- **Runtime key-recovery via emulation.** The RC4 key that never exists as contiguous bytes in RoningLoader's DLL (§6, item 7) is, in principle, recoverable by emulating its assembly routine (e.g. with Unicorn) rather than any static byte-pattern search, a materially larger engineering effort than the current static config-recovery module, and scoped out of this thesis accordingly.

Conclusion

This thesis presented MalWhere, a pipeline that combines static and dynamic malware analysis into confidence-scored, standards-compliant threat intelligence: MITRE ATT&CK technique mappings exported as STIX 2.1 bundles and live MISP events. Two mechanisms extend it beyond a conventional static-plus-dynamic design: a confidence reconciliation model that recognizes agreement both across sources and across technique specificity levels (§3.4), and a resubmission loop that correctly attributes a multi-stage sample's distributed capability to the component that actually exhibits it rather than collapsing it into the parent binary's own score (§3.5). Positioned against the tools surveyed in §2, the pipeline is not a replacement for a broad reputation aggregator such as VirusTotal, which no single-team research pipeline can match for raw submission-corpus scale; it is the step that comes after, turning that first-pass signal into a reconciled, per-technique, evidence-linked mapping for the sample that actually needs one.

Validated against three malware families with materially different architectures (a single-stage ransomware binary, a feature-rich .NET infostealer, and a four-component loader/rootkit/RAT chain) against ground truth built from function-level manual reverse engineering, the pipeline reached family-level F1 scores of 1.00, 0.89, and 0.86 respectively, with near-perfect precision throughout (§4). Those numbers are only as trustworthy as the process behind them, which is why this thesis places equal weight on the audit trail that produced them: 13 discrepancies between pipeline output and ground truth were individually traced to a specific, checkable cause, not accepted or dismissed on the aggregate score alone (§4.3); a real aggregation bug that had silently substituted the wrong analyzed binary was found and fixed (§5.2); and static detection coverage was extended from roughly 30 to approximately 85 MITRE technique identifiers, every addition sourced from MITRE's own technique descriptions and confirmed not to introduce new findings on the validated set before being trusted (§3.6).

The central argument of this work is that this audit trail, every rule tied to verifiable evidence, every discrepancy diagnosed to a specific cause, every extension checked for regression

before being kept, is a more defensible contribution than the headline F1 numbers by themselves, and is the property most likely to transfer when the pipeline meets a malware family it has not been tuned against. Precisely because that transfer has not yet been measured, expanding the validated family set (§7) is identified as the single highest-value next step, alongside the concretely-scoped extension to Linux/ELF-based IoT botnet analysis once its two missing prerequisites, real ELF-aware detection rules and a ground-truth Linux sample, are in place.

References

[1] MITRE Corporation, "MITRE ATT&CK." https://attack.mitre.org/, 2026. Enterprise Matrix, accessed 2026.

[2] MISP Project, "MISP: Open Source Threat Intelligence Platform." https://www.misp-project.org/, 2026.

[3] CAPE Sandbox Project, "CAPE: Config And Payload Extraction." https://github.com/kevoreilly/CAPEv2, 2026.

[4] Cuckoo Sandbox Project, "Cuckoo Sandbox: Automated Malware Analysis." https://cuckoosandbox.org/, 2022.

[5] Mandiant FLARE Team, "capa: The FLARE Team's Open Source Tool to Identify Capabilities in Executable Files." https://github.com/mandiant/capa, 2026.

[6] G. Husari, E. Al-Shaer, M. Ahmed, B. Chu, and X. Niu, "TTPDrill: Automatic and Accurate Extraction of Threat Actions from Unstructured Text of CTI Sources," in *Proceedings of the 33rd Annual Computer Security Applications Conference (ACSAC)*, 2017.

[7] MITRE Engenuity Center for Threat-Informed Defense, "TRAM: Threat Report ATT&CK Mapper." https://github.com/center-for-threat-informed-defense/tram, 2026.

[8] V. Legoy, M. Caselli, C. Seifert, and A. Peter, "Automated Retrieval of ATT&CK Tactics and Techniques for Cyber Threat Reports." https://arxiv.org/abs/2004.14322, 2020. arXiv:2004.14322.

[9] VirusTotal (Google), "VirusTotal." https://www.virustotal.com/, 2026.

[10] OASIS Cyber Threat Intelligence Technical Committee, "STIX Version 2.1." https://docs.oasis-open.org/cti/stix/v2.1/stix-v2.1.html, 2021.

[11] Mandiant FLARE Team, "FLOSS: FLARE Obfuscated String Solver." https://github.com/mandiant/flare-floss, 2026.

[12] MITRE Corporation, "MITRE ATT&CK STIX Data (Enterprise)." https://github.com/mitre/cti, 2026. `enterprise-attack.json`, accessed 2026.

Detection Rule Coverage Extension

Full technique-by-technique listing for the six coverage batches summarized in §3.6. Every row was verified to fire on none of the three validated samples after being added, confirming the extension does not alter Table 4.

Technique	Name	Detection mechanism
Batch 1: Persistence & Defense Evasion		
T1546.010	AppInit DLLs	`AppInit_DLLs` registry string
T1547.004	Winlogon Helper DLL	`Winlogon\Shell`/`Userinit` string

continued on next page

Technique	Name	Detection mechanism
T1547.005	Security Support Provider	`Security Packages` string
T1547.002	Authentication Package	`Authentication Packages` string
T1546.012	Image File Execution Options Injection	`Image File Execution Options` string
T1546.003	WMI Event Subscription	`__EventFilter`/`__EventConsumer`/ `__FilterToConsumerBinding` strings
T1197	BITS Jobs	`bitsadmin` string
T1070.001	Clear Windows Event Logs	`wevtutil cl` string
T1218.010	Regsvr32 (Squiblydoo)	`scrobj.dll` string
T1218.005	Mshta	`mshta.exe` + `.hta` (both required)
T1055.012	Process Hollowing	combo (§3.2, Table 1)
Batch 2: Credential Access & Collection		
T1003.001	LSASS Memory	`MiniDumpWriteDump` import + `lsass.exe` string
T1003.002	Security Account Manager	`hklm\sam` string
T1003.004	LSA Secrets	`SECURITY\Policy\Secrets` string
T1552.002	Credentials in Registry	registry credential-path patterns
T1552.004	Private Keys	PEM/OpenSSH markers, `.ppk`
T1555.004	Windows Credential Manager	`VaultOpenVault` import, `VaultCli.dll` string
T1556.002	Password Filter DLL	`Notification Packages` string
T1040	Network Sniffing	`wpcap.dll`/`Packet.dll` strings
T1125	Video Capture	`avicap32.dll` string
T1114.001	Local Email Collection	`.ost`/`.pst` strings
T1039	Data from Network Shared Drive	`WNetOpenEnumW/A` + `WNetEnumResourceW/A` + `WNetCloseEnum`
Batch 3: Execution & Command and Control		
T1053.005	Scheduled Task	`schtasks` string / command pattern
T1059.001	PowerShell (encoded/hidden)	`-EncodedCommand` + `-WindowStyle Hidden` (both required)
T1559.002	Dynamic Data Exchange	`DDEAUTO` string
T1127.001	MSBuild	`msbuild.exe` string
T1569.002	Service Execution	`sc start`/`net start` strings
T1102.002	Web Service (Bidirectional)	`api.telegram.org`, `pastebin.com/raw`, `raw.githubusercontent.com`, `discord.com/api/webhooks`
T1572	Protocol Tunneling	`Renci.SshNet`, `ngrok`, `serveo.net`
T1090.002	External Proxy	`:9050` (Tor SOCKS port) string
Batch 4: Discovery		
T1518.001	Security Software Discovery	`MsMpEng.exe` / `avp.exe` / `SecurityCenter2` strings
T1049	System Network Connections Discovery	`GetTcpTable` / `GetExtendedTcpTable` / `GetUdpTable` imports
T1087.001	Local Account Discovery	`NetUserEnum` import
T1018	Remote System Discovery	`NetServerEnum` import
T1120	Peripheral Device Discovery	`SetupDiGetClassDevsW/A` import
T1614.001	System Language Discovery	`GetSystemDefaultLangID`/ `GetUserDefaultUILanguage` imports
T1010	Application Window Discovery	combo (§3.2, Table 1)
T1033	System Owner/User Discovery	`GetUserNameW/A` import (moved from T1082)
Batch 5: Impact		
T1529	System Shutdown/Reboot	`ExitWindowsEx` import
T1531	Account Access Removal	`NetUserChangePassword` import
T1561.001	Disk Content Wipe	`\\.\PhysicalDrive` string
T1496	Resource Hijacking	`stratum+tcp://` string
Batch 6: Privilege Escalation & Exfiltration		
T1134.001	Token Impersonation/Theft	`ImpersonateLoggedOnUser` + `DuplicateToken(Ex)`
T1134.002	Create Process with Token	`CreateProcessWithTokenW` import
T1134.003	Make and Impersonate Token	`ImpersonateLoggedOnUser` + `LogonUserW/A`
T1548.002	Bypass User Account Control	`ms-settings` \ `Shell` \ `Open` \ `command` string
T1546.008	Accessibility Features	`sethc.exe` string
T1546.009	AppCert DLLs	`AppCertDlls` string
T1547.014	Active Setup	`Active Setup\Installed Components` string
T1098	Account Manipulation	`net localgroup administrators` string
T1567.001	Exfil. to Code Repository	`api.github.com` string
T1567.002	Exfil. to Cloud Storage	`dropboxapi.com` / `storage.googleapis.com` strings
T1567.003	Exfil. to Text Storage Sites	`pastebin.com` / `api` / `api_post.php` string
T1567.004	Exfil. over Webhook	dual-mapped from T1102.002 Discord webhook evidence
T1048.003	Exfil. over Unencrypted Non-C2 Protocol	`FtpPutFileW/A` import

continued on next page

Technique	Name	Detection mechanism

Table 5: 55 technique identifiers added across six coverage batches (§3.6), sourced from MITRE's own per-technique descriptions.